

Université Ibn Zohr

Ecole Supérieur de technologie Agadir

Rapport de Stage de fin d'étude

Réalisé par :

IBRAHIM ID-WAHMAN

Filière

2ème année Ingénierie de données

Au sein de la société :



Cabinet Vecteurs

4ème étage, Bureau N° BB401, Technopole 2, Bloc B

Période de stage : 2 mois

Stage de Fin d'Etude (SFE)

Réalisé par
Ibrahim Id-wahman

Membre de jury

M. Y.El Ansari

M. Amrouch

M. Hachimi

Encadré par :
M. Y.El Ansari

Réalisé le
01/06/2026

Remerciement

Je tiens avant tout à exprimer ma reconnaissance à Mme Rifah et pour avoir accepté de m'encadrer dans cette étude. Je la remercie pour son implication, son soutien et ses encouragements tout au long de ce travail.

Je souhaite également remercier M.Y.El Ansari pour avoir su me faire confiance.

Merci.

Table des matières

Liste des tableaux	2
Liste des figures	3
Résumé	4
Abstract	5
Glossaire	6
Introduction	8
Chapitre 1 — Présentation de l'entreprise et du service d'accueil	10
1. Présentation générale de l'entreprise	10
1.1. Historique et évolution récente :	10
1.2. Fiche technique :	10
1.3. Secteur d'activité :	10
1.4. Organigramme général :	12
1.5. Principaux partenaires :	12
1.6. Les clients :	12
2. Présentation de service d'accueil	13
2.1. Mission et rôle du service	13
2.2. Outils et logiciels utilisés	13
3. Présentation de service d'accueil	14
3.1. Situation initiale observée à l'arrivée	14
3.2. Besoin exprimé et difficulté constatées	14
Chapitre 2 — Introduction au concepts de l'apprentissage profond (deep learning)	15
1. Modèle de langage (Language model Anglais)	15
2. Large language models (LLMs Anglais)	15
2.1. Réseaux neuronal récurrent (RNN) :	15
2.2. Le n-gram des mots :	15
2.3. Transformers :	16
3. RAG (Retrieval Augmented Generation) :	16
3.1. Définition :	16
3.2. Comment ça fonctionne ?	17
3.3. Le processus du modèle :	17
3. Fine Tuning	18
4. Concepts bases de RAG :	18
4.1. Vectors database (Base de données des connaissances)	18
4.2. Le model pré entraîné :	18
4.3. Prompt Engineering :	18
4.4. Lang Chain :	19
1. Contexte du chatbot RAG :	20
2. Analyse de l'existant :	20
3. Technologies utilisées	20
4. Introduction à data.gov.ma	21
5. Avant RAG :	21
5.1. Paramètres du modèle :	22
5.2. Préparation de structure de projet :	22
5.3. Implémentation du modèle :	23
6. Processus	27
7. Conclusion	28
Chapitre 2 — Mise en place de moteur d'indexation et de recherche	29
1. Comment ça fonctionne 'Lucene' en tant qu'indexeur de fichiers ?	29
2. Technologies et Matériels :	29
3. Exigences fonctionnels (de la part de l'utilisateur) :	30
Annexes	32
Partie 3 — Mise en place d'une application de gestion de projet	34
1. Contexte et technologies	34



1.1. ElectronJs :	34
1.2. ORM (Object Relationnal Mapping)	34
1.3. Chartjs :	35
2.Fonctionnalités	35
2.1. Visualisation & tableau de bord :	35
2.2. Diagramme de Gantt :	36
2.3. Tableau des tâches Kanban :	37
2.4. Création et gestion des tâches :	38
2.5. Editeur de texte :	40
2.6. Uploader des fichiers :	41
4. Architecture du Système	42
4.1 Vue d'Ensemble	42
4.2. Composantes Principales	42
4.3. Spécifications Technique	42
Backend :	43
Frontend :	43
6. Fonctionnalités Implémentées	43
6.1 Authentification et Gestion Utilisateur	43
6.2 Gestion de Projets et Tâches	43
6.3 Édition Avancée de Tâches	43
6.4 Collaboration d'Équipe	44
6.5 Dashboard et Profil Utilisateur	44
6.5 Diagramme de Gantt :	44
7. Cas de Tests :	44
8. Défis Techniques	45
Récapitulatif	46
1. Réalisations	46
1. Perspectives et Améliorations Futures	46
Conclusion de la partie	47
Conclusion générale	48
Bibliographie	49

Liste des tableaux

Table 1 Suivie géographique du Cabinet VECTEURS	10
Table 2 Fiche technique du cabinet VECTEURS	10
Table 3 Pôle d'activité de Cabinet VECTEURS	11
Table 4 Tableau descriptif des technologies	44
Table 3 Tableau des défis techniques de l'application bureau	47

Abstract

Large Language Models (LLMs) are advanced models having the ability to understand human language by implementing Natural Language Processing (NLP) which give them the power to understand complex questions and extract knowledge from documents. In this report I have described three projects I made while passing my internship, the most project who took most of my time was the chat bot assistant, because it needed a lot of reasoning and analysis to choose between two methods to make a large language model answers questions based on the knowledge they have (documents, per-training data, internet tools & resources...). The second one was an file indexer engine, to make file search easier and fast, instead of struggling to remember where the file was, or using file explorer to search for name matching, instead it reads the content and use it to build a set of keywords for that file, to allow search with keyword, also can search using sub string of file content, all that managed by **Apache Lucene** a fast search engine. The last project was making a project manager app used to manage projects and tasks, I used **ElectronJS** as a desktop app engine that launches the app (GUI), **Flask** as a service engine to manage routes and files also to configure the database (SQLite), it includes a table for kanban tasks, Gantt diagram, file management and dashboards of statistics. This project helps me to enhance my skills at desktop application developpement by exploring new concepts like ipcRenders, Internal API..., and UI/UX as a desktop application responsive design.

Glossaire

API : Représente le point de liaison entre l'interface utilisateur et le serveur, pour garantir que les données affichées seront dynamiques, réelles et s'échangent en même temps.

RNN : abréviation de Recurrent Neural Network and anglais, c'est un réseau neuronal avancés qui a la capacité de rappeler des sorties du nœud précédent.

Flow : « flux » en français, est le passage d'une étape à une autre.

KPI : Indice de performance utilisé notamment dans les tableaux de bords, pour aider à prendre des décisions

Tika : Framework d'Apache qui donne la possibilité d'extraire les informations d'un fichier quel que soit son type inclure les métadonnées, le texte.

Lucene : Framework de Apache qui joue le rôle d'un moteur de recherche libre et open source, largement utilisé dans les applications de recherches et les systèmes standard.

RAG : C'est de donner des ressources à un modèle déjà entraîné, et assurer qu'il consulte ces ressources avant de répondre à chaque question.

Fine Tunning : C'est le processus d'adaptation des modèles déjà entraînés à un ensemble de données petit, pour assurer que le modèle soit capable de faire une tâche précise.

LLm (Large language model) : c'est un modèle de langage plus complexe, qui a la possibilité de comprendre, générer et analyser les données humaines (textes, images, audio, ...).

GPU (Graphical Process Unit) : C'est un circuit électronique spécialisé pour les opérations de calculs énormes et complexes, aussi dans les opérations graphiques qui nécessitent une grande performance tant que les jeux, traitement des images et des vidéos, entraînement des modèles...[\[source\]](#).

C : Language de programmation créée en 1970 par Dennis Retchie, il est connu par sa flexibilité et son accès aux ressource externes d'un périphérique, généralement utilisée pour créer les noyaux des systèmes, les [drivers](#).

Nodejs : C'est un environnement d'exécution des projets Javascript, qui fonctionne sur tous les systèmes (Windows, Linux, Mac, ...)

ORM : abrev de 'Object Relationnal Mapping' c'est une méthode qui consiste à manipuler les requêtes des bases de données sous forme de fonctions et classes en OOP. [\[Voir\]](#)

BERT : abrev de 'Bidirectionnal Encoder Representations Transformer', c'est une architecture qui permet d'améliorer la performance et la précision de compréhension de contexte et de tokenisation des phrases complexes.

GUI : abrev de Graphical User Interface, est un terme utilisé pour décrire les interface utilisateur au niveau des applications bureau (desktop apps) [\[voir\]](#).

Introduction

L'ingénierie de données est la discipline de mettre en place la configuration et la préparation des systèmes de gestion d'agrégation, de stockage et d'analyse de données à grand échelle, elle importe aussi le domaine de construction des programmes / outils d'automatisation du traitement de données, notamment ce qu'on appelle 'pipeline', qui signifie l'enchaînement d'instructions qu'un système de traitement de données automatisées va exécuter pour traiter des données volumineuses et avoir un résultat qui satisfait les besoins.

Pour créer ce genre de systèmes , il faut suivre ces étapes :

- **Déterminer le besoin et préparer les ressources :** Transporter les données à traiter vers un environnement isolé, déterminer les besoins (le pourquoi), et les données ressources (*input/outputs*), c'est à dire qu'elles sont les entrées que ce système va utiliser de ces données ? et qu'ils sont les données sorties qu'on a besoin ?
- **Créer l'échantillon et analyser :** Avant de commencer le traitement sur tous les données, on prend un échantillon de ces données pour appliquer le traitement, on doit choisir un type effectif d'échantillonnage selon la structure de nos données, généralement il est recommandé d'utiliser les différents types d'échantillonnage pour garantir la dispersion, et que l'échantillon contient presque tous les différents types de nos données.
Après créer notre échantillon, on va appliquer nos algorithmes pour évaluer la qualité des résultats, et aussi choisir l'algorithme de traitement.
- **Préparer le matériel de traitement :** Après vérifier la qualité d'échantillonnage et le choix de l'algorithme de traitement (l'ordre des étapes de traitements), on prépare le système qui va traiter ces données et on le met dans un **matériel informatique (hardware)**, tenant en compte la durée du traitement, les ressources matériels (électricité, carte graphique si besoin,).

Chapitre 1 — Présentation de l'entreprise et du service d'accueil

1. Présentation générale de l'entreprise

1.1. Historique et évolution récente :

Créée au Maroc en 2003, le cabinet Vecteurs constitue un fruit ; à la fois ; de l'expérience française et tunisienne, tout en ayant l'art de s'adapter au contexte marocain typique afin de mieux combler le besoin du marché local.

1988 à 1993	Cabinet VECTEURS France
1993 à 2003	Cabinet VECTEURS Tunisie
2003 à aujourd'hui	Cabinet VECTEURS Maroc

Table 1 Suivre géographique du Cabinet VECTEURS

1.2. Fiche technique :

RAISON SOCIALE	Cabinet VECTEURS
STATUT JURIDIQUE	S.A.R. L
DATE DE CREATION	2003
DOMAINE D'ACTIVITE	Bureau d'études économique, sociales et environnementales.
DIRIGEANT	Brahim EL IBRAHIMI
ADRESSE	4ème étage, Bureau N° BB401, Technopole 2, Bloc B, Agadir 80000
CAPITAL SOCIAL	1.000.000,00 DH
TELEPHONE	+212 5 28 23 97 25
FAX	+212 5 28 21 57 72
EMAIL	contact@cabinetvecteurs.com
SITE WEB	www.cabinetvecteurs.com

Table 2 Fiche technique du cabinet VECTEURS

1.3. Secteur d'activité :

Le cabinet Vecteurs intervient dans trois grands pôles d'activité, à savoir : la réalisation des études, le conseil et l'accompagnement des entreprises ainsi que l'organisation des formations et des programmes de renforcement des capacités.

Table 3 Pôle d'activité de Cabinet VECTEURS

2. Présentation de service d'accueil

2.1. Mission et rôle du service

Le Cabinet Vecteurs a pour mission principale de fournir des prestations de qualité à ses clients entreprises. Pour ce faire, il assure plusieurs missions complémentaires : la sensibilisation des collaborateurs aux nouveaux projets, l'écoute des besoins des entreprises afin de leur proposer des services adaptés, l'implication des parties prenantes dans leurs projets de développement, l'assistance jusqu'à la réussite de leurs projets, l'accompagnement dans la conduite du changement, ainsi que l'amélioration des performances à travers des stratégies de mise à niveau.

Le service au sein duquel j'ai effectué mon stage est directement responsable de la gestion et du suivi des formations proposées aux entreprises clientes. Son rôle couvre l'ensemble du cycle de vie d'une formation, depuis la réception de la demande client jusqu'à la remise des documents finaux. Concrètement, lorsqu'une entreprise contacte le cabinet pour une formation, le service prend en charge la préparation de la fiche technique, la désignation du formateur, la définition du lieu et du calendrier de la session. À l'issue de la formation, il assure l'établissement de l'ensemble des documents administratifs nécessaires : facture, liste de présence et fiche d'évaluation.

2.2. Outils et logiciels utilisés

Durant mon stage au Cabinet Vecteurs, j'ai pu constater que le service s'appuie essentiellement sur des outils bureautiques classiques pour gérer son activité formation. Microsoft Word est utilisé pour la création et la mise en forme des documents administratifs tels que les fiches techniques, les attestations de participation et les convocations. Microsoft Excel, quant à lui, sert au suivi des formations, à la gestion des données des participants ainsi qu'à l'établissement des factures. Microsoft PowerPoint est utilisé par les formateurs pour la préparation et la présentation des supports pédagogiques lors des sessions de formation.

3. Présentation de service d'accueil

Dès mon premier jour au Cabinet Vecteurs, j'ai eu un entretien avec mon encadrante professionnelle Mme. Dahbi Rifah, qui m'a présenté le fonctionnement général du cabinet et le contexte de ma mission. Elle m'a expliqué que l'équipe consacrait une part importante de son temps à des tâches administratives répétitives, notamment la saisie manuelle des mêmes informations dans plusieurs documents distincts pour chaque formation organisée. Afin de me permettre de comprendre concrètement le problème, elle m'a remis une clé USB contenant l'ensemble des documents existants (fiches techniques, factures, listes de présence et fiche d'évaluation...) pour que je puisse analyser les données et identifier précisément les besoins.

Après analyse des documents fournis, j'ai pu constater que les mêmes informations (nom de l'entreprise cliente, intitulé de la formation, dates, lieu, et liste des participants) étaient ressaisies manuellement dans chaque document séparément. Mme. Dahbi Rifah m'a confirmé que ce processus engendrait deux problèmes majeurs : une perte de temps considérable qui mobilisait l'équipe sur des tâches à faible valeur ajoutée, et des risques fréquents d'erreurs de saisie pouvant affecter la qualité des documents remis aux clients.

Chapitre 2 — Introduction au concepts de l'apprentissage profond (deep learning)

1. Modèle de langage (Language model Anglais)

Un modèle de base informatique au sein du NLP (Natural Language Processing), ce sont des modèles qui ont la capacité de lire et comprendre les données humaines séquentielles et les utiliser dans l'entraînement, pour produire des données similaires (y compris *la génération de texte, la reconnaissance vocale*).

2. Large language models (LLMs Anglais)

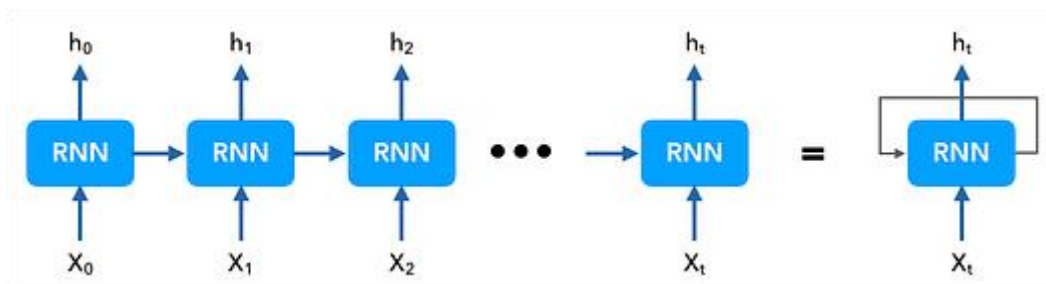
Un modèle de langage étendu (Large Language model) est une forme complexe de modèle de langage qui peut comprendre les structures complexes des données et met en œuvre des algorithmes avancés pour améliorer les performances des prédictions. Dans le cas de données séquentielles, il existe des concepts majeurs :

2.1. Réseaux neuronal récurrent (RNN) :

un réseau neuronal qui traite des données séquentielles telles que le texte, la parole et les séries temporelles, il utilise des algorithmes d'apprentissage automatique pour former la relation entre les entrées (inputs), et les sorties (outputs), il se différencie des autres types de réseaux neuronaux car il comporte une mémoire.

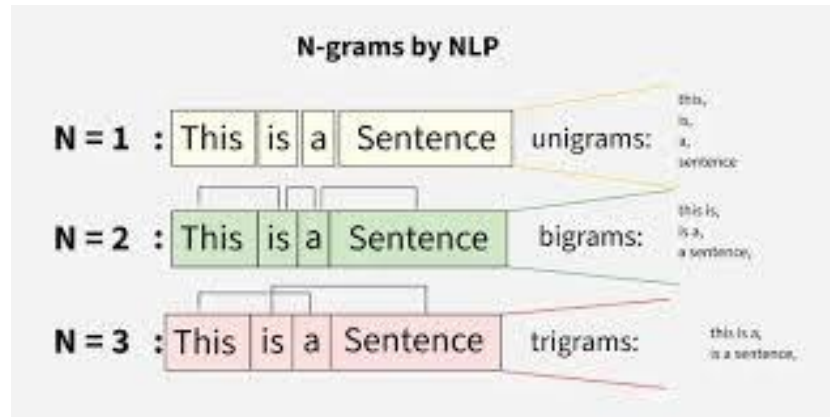
Le schéma ci-dessous montre la structure d'un réseau RNN, dont les $\mathbf{x}_i$ représentent la donnée entrée à l'instant 'et', après avoir la traiter, le nœud conserve la valeur ' h_i ' qui représente l'information extraite lors du traitement de celle précédente.

Ça l'aide à rappeler du contexte du mot précédente et obtenir le maximum de l'information.



2.2. Le n-gram des mots :

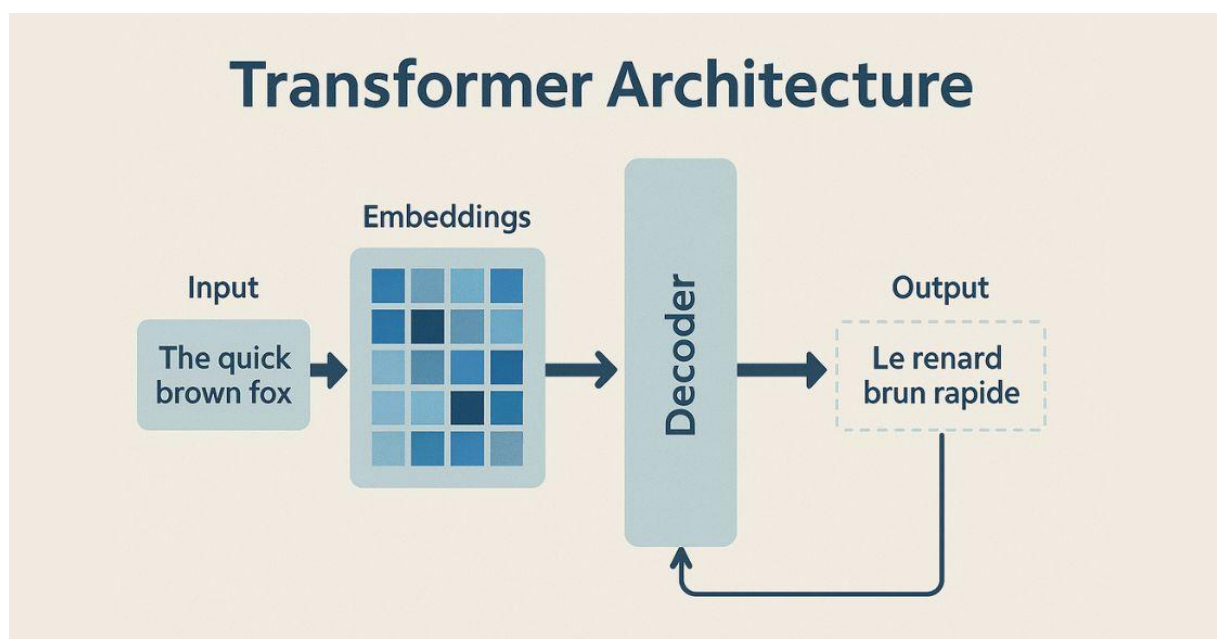
Représente le calcul de la probabilité du mot suivant en fonction des coefficients des n mots précédentes.



2.3. Transformers :

Représentent l'événement déclencheur qui a poussé les GPT (modèles qui peuvent générer des données humaines et qui ont la capacité de réfléchir et raisonner) à être populaire et capable de faire presque toutes les tâches humaines.

Le schéma ci-dessous représente la structure basique d'un transformer Anglais



3. RAG (Retrieval Augmented Generation) :

3.1. Définition :

RAG est le fait d'optimiser la performance d'un modèle d'intelligence artificielle, en créant la connexion avec des connaissances externes (base de données des connaissances, entrepôt, ...), RAG aide les LLMs (Large Language model) à délivrer des réponses plus précises et qui ont une forte relation avec le contexte de donnés.

18

Par exemple, on peut utiliser le prompt engineering pour demander à un modèle de langage de résumer un texte, de traduire une phrase, de répondre à une question, etc. Le prompt engineering nécessite de comprendre les capacités et les limites du modèle de langage, ainsi que les besoins et les attentes de l'utilisateur. Sans avoir besoin de connaissances approfondies en programmation ou en apprentissage automatique. Il implique plusieurs techniques :

- **Few-shot Learning** : Il s'agit d'utiliser un petit nombre de prompt soigneusement conçues pour activer un modèle de langage pour effectuer une nouvelle tâche ou s'adapter à un nouveau domaine [2].
- **Chain-of-thought prompting** : En utilisant une séquence de prompt interconnectées pour guider les réponses du modèle de langage de manière cohérente et logique [3].
- **Self-consistency** : Utiliser des prompts pour encourager le modèle de langage à générer des réponses qui correspondent à ses réponses précédentes [4].

4.4. Lang Chain :

C'est l'interface générique qui représente la base des opérations RAG, il permet de connecter un modèle de langage profond (LLM) et le donner l'accès aux ressources internes,

- 21

```
prompt = "is the earth flat ?"

inputs = tokenizer(prompt, return_tensors="pt")
outputs = model.generate(**inputs, max_new_tokens=100)

print(tokenizer.decode(outputs[0], skip_special_tokens=True))

... The idea that the Earth is flat is a misconception that has been widely debunked by scientific evidence. For centuries, observations of the horizon, and the way ships and airplanes navigate around the globe. The overwhelming consensus among the scientific community is that the Earth is not flat, but round, based on extensive research and empirical data.
```

Comme on peut voir, le modèle fonctionne pour ce genre de questions, ce qui signifie qu'il est adapté à ces questions en anglais – en français :

```
prompt = "est ce que la Terre est plate?"

inputs = tokenizer(prompt, return_tensors="pt")
outputs = model.generate(**inputs, max_new_tokens=100)

print(tokenizer.decode(outputs[0], skip_special_tokens=True))

a Terre est plate est une conception erronée largement réfutée par les preuves scientifiques. Depuis des siècles,
```

Donc le test est vérifié, après évaluer la probabilité que le modèle a la capacité de comprendre les questions (anglais et français), on va passer vers l'étape suivante, c'est de préparer les données que le modèle va récupérer, et d'extraire de l'information valide et importante d'après les fichiers/données qu'on a. Les données vont être :

- Informations sur l'historique de l'entreprise, les activités...
- Autres informations sur des différentes régions du Maroc, et des divers Thèmes comme 'finance', société, agriculture...

5.1. Paramètres du modèle :

Ce sont les paramètres pour configurer le modèle lors de ses réponses, ce qui concerne le ton et la manière de répondre (professionnel,)

- **Tone** : Le ton d'un chatbot désigne la manière de qu'il répondre, son attitude, que le modèle va utiliser pour s'interagir avec le modèle. Il détermine comment le modèle va communiquer avec l'utilisateur, soit d'être amical, professionnel ou formel.
-

5.2. Préparation de structure de projet :

Le projet va prendre une structure organisée sous la forme suivante :

- **/app** : contient l'interface et l'API que l'utilisateur (non dev) va s'interagir avec, il représente le côté serveur qui va s'interagir directement avec le model.
- **/data** : représente les données brutes qui vont être transformées après.

- ```

 graph TD
 RAG_ASSISTANT[▼ RAG ASSISTANT]
 RAG_ASSISTANT --> app[>  app]
 RAG_ASSISTANT --> data[>  data]
 RAG_ASSISTANT --> rag[▼  rag]
 rag --> embeddings[▼  embeddings]
 embeddings --> embedder_py[ embedder.py]
 embeddings --> models_py[ models.py]
 embeddings --> generation[>  generation]
 embeddings --> ingestion[>  ingestion]
 embeddings --> retrieval[>  retrieval]
 embeddings --> vectorstore[>  vectorstore]
 embeddings --> pipeline_py[ pipeline.py]
 rag --> scripts[>  scripts]
 rag --> tests[>  tests]
 rag --> env[ .env]
 rag --> makestructure_py[ makestructure.py]
 rag --> README_md[ README.md]
 rag --> requirements_txt[ requirements.txt]

```

Figure 4 : Structure d'un projet RAG

### 5.3. Implémentation du modèle :

### 5.3.1. Question Answering :

Le chatbot qu'on va implémenter, et qui va répondre aux questions, va être exécuté au niveau de l'espace [Google Colab](#), et en utilisant des modèles de [Hugging Face](#).

```
#Installation de toutes les bibliothèques nécessaires
!pip -q install langchain
!pip -q install bitsandbytes accelerate xformers einops
!pip -q install datasets loralib sentencepiece
!pip -q install pypdf
!pip -q install sentence_transformers
!pip install chromadb
!pip install openai
!pip install tiktoken

#Importation des fonctions nécessaires
from langchain.document_loaders import PyPDFLoader
from langchain.document_loaders import TextLoader
from langchain.document_loaders import Docx2txtLoader
from langchain.text_splitter import CharacterTextSplitter
from langchain.embeddings import HuggingFaceEmbeddings
from langchain.vectorstores import Chroma
from huggingface_hub import notebook_login
import torch
import transformers
from transformers import AutoTokenizer, AutoModelForCausalLM
from transformers import pipeline
from langchain import HuggingFacePipeline
from langchain.chains import ConversationalRetrievalChain
from langchain.memory import ConversationBufferMemory
from langchain.embeddings.openai import OpenAIEmbeddings
from langchain.chat_models import ChatOpenAI
import os
import sys
```

L'étape suivante est de charger les documents qu'on va utiliser pour les donner au modèle :

Le script commence par créer un dossier "docs" et y charge des documents PDF, DOC/DOCX, et TXT à partir de ce dossier. Ces documents sont chargés à l'aide de différentes classes de chargement, selon leur format.

```
#Importation des documents
!mkdir docs

#Itération sur tous les documents importés en vérifiant les extensions afin de bien charger le contenu dans la liste "document"
document=[]
for file in os.listdir("docs"):
 if file.endswith(".pdf"):
 pdf_path="./docs/"+file
 loader=PyPDFLoader(pdf_path)
 document.extend(loader.load())
 elif file.endswith('.docx') or file.endswith('.doc'):
 doc_path="./docs/"+file
 loader=Docx2txtLoader(doc_path)
 document.extend(loader.load())
 elif file.endswith('.txt'):
 text_path="./docs/"+file
 loader=TextLoader(text_path)
 document.extend(loader.load())
```

- **Division des documents et fichiers en « chunks » :** On va utiliser *Apache Tika* pour extraire les informations d'après les fichiers PDF et WORD, et ensuite on va les diviser en morceaux :

25

```
import torch
Tokeniser les données d'entraînement
tokenized_train = tokenizer(train_texts, return_tensors="pt", padding=True, truncation=True)
train_inputs = tokenized_train["input_ids"]
train_labels = torch.tensor(train_labels)
```

### c. Data Loader :

Un Data Loader est une classe qui facilite l'itération d'importer les 'parties' « chunks » de documents à notre modèle.

```
from torch.utils.data import DataLoader, TensorDataset

Créer un DataLoader pour les données d'entraînement
train_dataset = TensorDataset(train_inputs, train_labels)
train_dataloader = DataLoader(train_dataset, batch_size=2, shuffle=True)
```

#### d. Training Loop (Boucle d'entraînement) en Fine Tunning :

C'est l'opération qui se répète **n** fois (**n** = nombre d'époques), qui représente le cycle d'entraînement de modèle et à chaque itération, la classe **DataLoader** choisit la partie sélectionnée pour le nombre d'itération.

```
Boucle d'entraînement
for epoch in range(epochs):
 model.train() # Mettre le modèle en mode d'entraînement

 for batch in train_dataloader:
 inputs = batch[0]
 labels = batch[1]

 optimizer.zero_grad() # Remettre à zéro les gradients
 outputs = model(inputs, labels=labels) # Propagation avant
 loss = outputs.loss # Calcul de la perte
 loss.backward() # Rétropropagation des gradients
 optimizer.step() # Mise à jour des poids
```

e. Fonction de classification de texte (professionnel, non professionnel) :

Utiliser BERT pour prendre un texte comme entrée, le tokenize, et décider si un texte est professionnel ou pas, en donnant la valeur calculée de probabilité

(threshold).

```
import torch
from transformers import BertTokenizer, BertForSequenceClassification

def classify_text(text, model, tokenizer, threshold=0.5):
 # Tokenisation du texte
 inputs = tokenizer(text, return_tensors="pt")

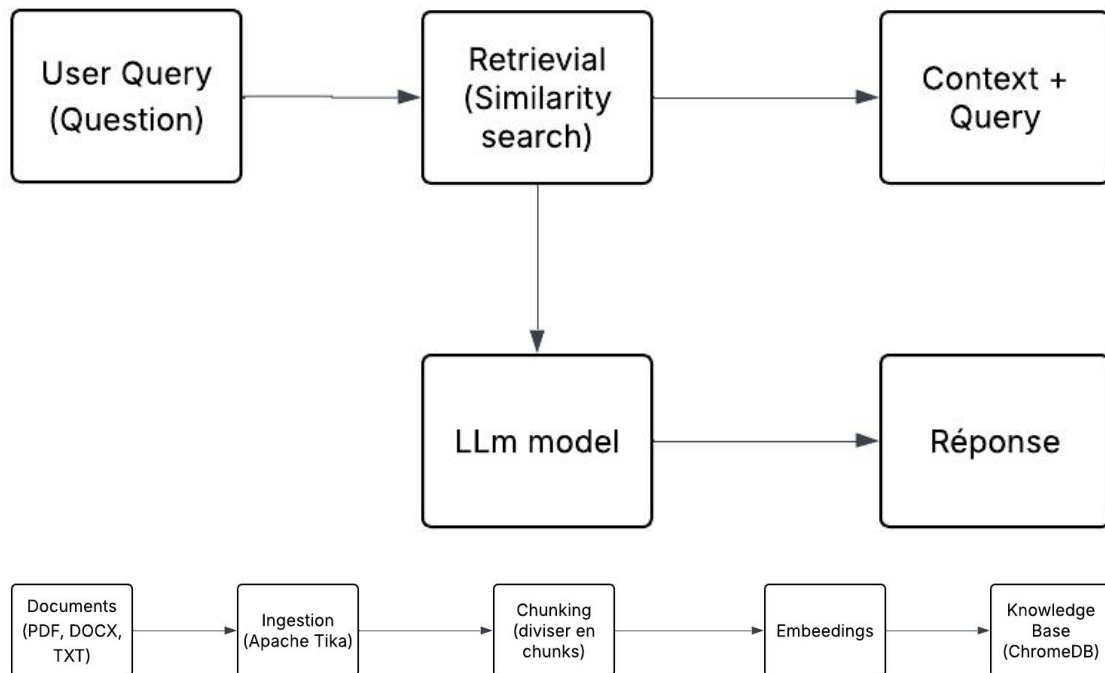
 # Prédiction de la classe
 with torch.no_grad():
 logits = model(**inputs).logits

 # Application de la fonction softmax pour obtenir les probabilités
 probabilities = torch.softmax(logits, dim=1)

 # Vérification si la probabilité de texte non professionnel dépasse le seuil
 non_professional_probability = probabilities[0, 1].item()
 non_professional_detected = non_professional_probability > threshold

 return non_professional_detected, non_professional_probability
```

## 6. Prozess



Ce diagramme montre le flux complet de votre système d'IA. Les documents sont d'abord traités par Apache Tika, divisés en chunks, puis convertis en vecteurs d'Embeddings. Ces vecteurs sont stockés dans Chromadb. Quand un utilisateur pose une question, le système récupère les contextes pertinents de la base de connaissances et les combine avec la question avant de les envoyer au modèle Mistral 7B pour générer une réponse.

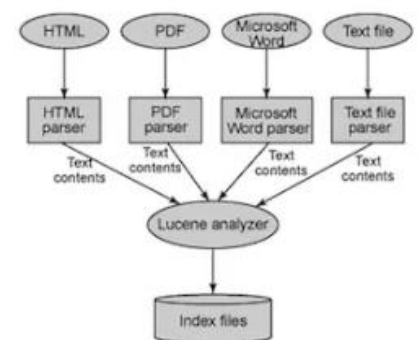


## Chapitre 2 — Mise en place de moteur d’indexation et de recherche

Un moteur de recherche rapide et puissant pour fichiers non organisés sur votre appareil, utilisant Lucene comme moteur d'indexation. Il indexe le nom des fichiers, leurs mots-clés et leur contenu, permettant une recherche rapide à travers plusieurs répertoires.

## 1. Comment ça fonctionne ‘Lucene’ en tant qu’indexeur de fichiers ?

Quand on importe un fichier dans Lucene, il passe par deux étapes, la première est de diviser ces fichiers vers des parties contenant des ‘mots clés’, ce qu’on appelle ‘chunks’, et pour chacun de ces ‘chunks’ on va le donner son propre index, si on cherche d’un mot qui a des correspondances dans d’un fichier source, il le prélève et le résultat.



## 2. Technologies et Matériels :

Avant de participer au développement de l'application, il faut comprendre tout d'abord toutes les exigences, techniques et fonctionnelles. Car les exigences ont des influences durant toutes les phases de développement.

Le projet est construit par plusieurs technologies (Framework), dont on a :

- **Apache Tika** : Framework open source créée par Java, et développée par Apache pour la détection du contenu et extraction des métadonnées et le texte à partir de plus que 1000 types d'extensions de fichiers, connue par sa performance et sa précision dans l'extraction du texte d'après la majorité des documents & des fichiers.



- **Apache Lucene** : Framework développée par Apache consiste à simplifier les opérations de recherche et d'indexer du texte elle est utilisée dans certains moteurs de recherches standards, caractérisée par sa performance (une requête dans sql5 prend 18



min et plus, tant que dans Lucene il prend moins qu'une second [6]) et l'utilisation réduit les ressources matérielles.

- **JSON** : JavaScript Object Notation, c'est la forme de données la plus utilisée pour conserver les données structurées et complexes, aussi qu'aide dans les opérations d'écriture et de lecture (I/O), lisible par l'humain, composé de paires nom-valeurs et supporte les tableaux (arrays).
- **CustomTkinter** : c'est l'équivalent de tkinter mais avec des designs modernes
- 



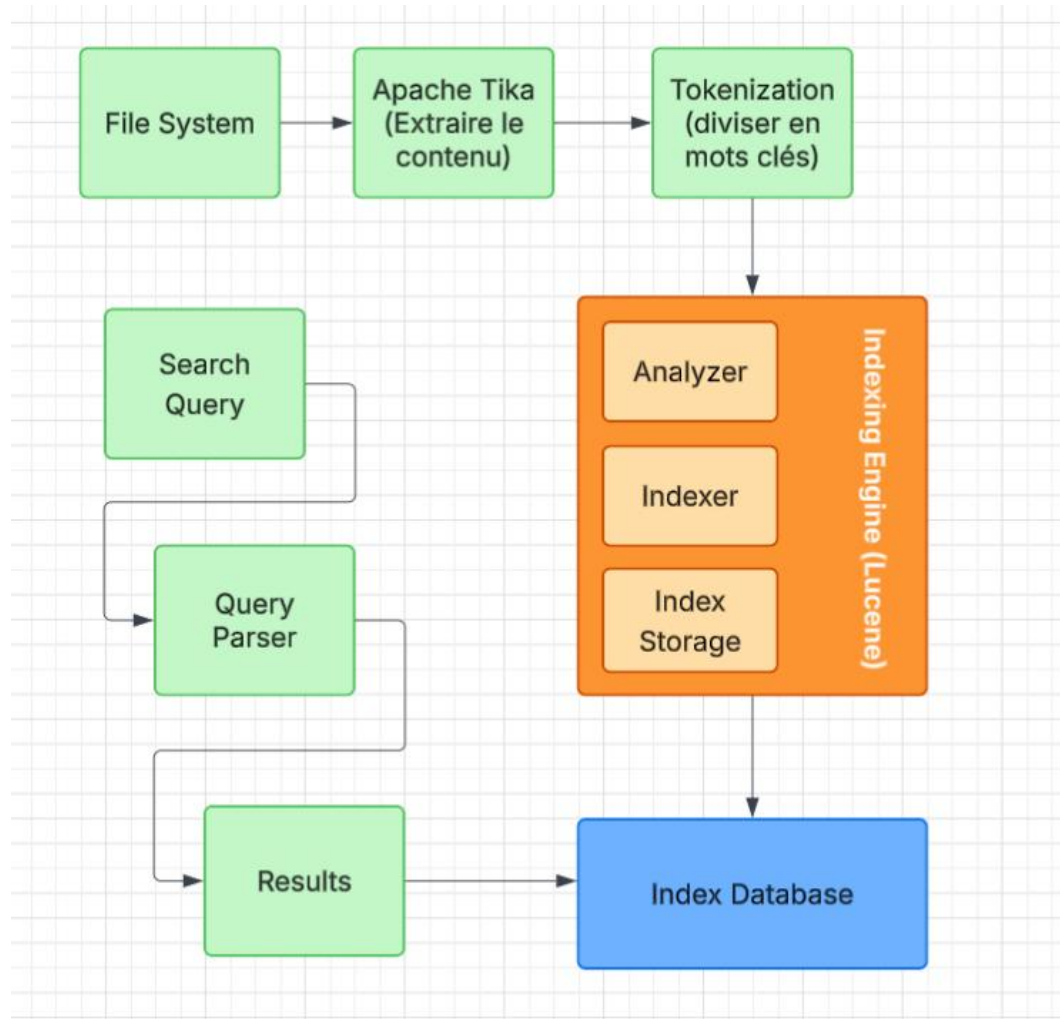
### 3. Exigences fonctionnels (de la part de l'utilisateur) :

C'est l'ensemble des fonctionnalités que le système offre pour l'utilisateur afin de satisfaire son besoin, dans ce cas on a :

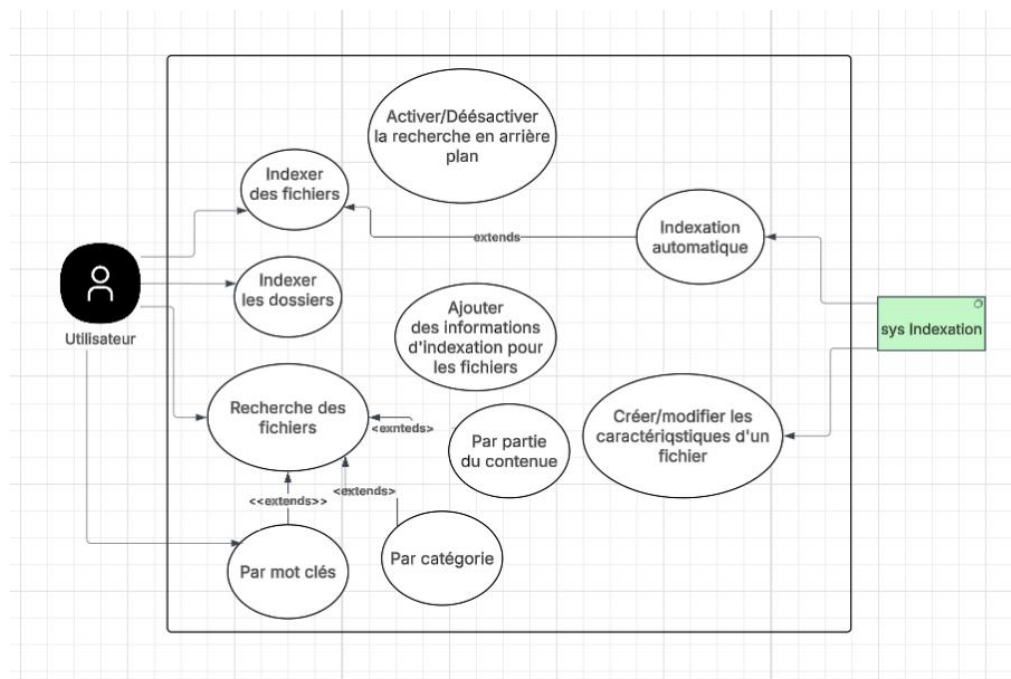
- Création des *indexeurs* (pointeurs qui indexent vers le fichier spécifiée) ;
- Ajouter un fichier de données à l'étude ;
- Chercher un fichier par plusieurs critères :
  - Nom.
  - Type (extension).
  - Mot clés.
  - Morceau de texte identique aux mots du contenu.
- Ajouter des fichiers personnalisés.
- Activer/Désactiver l'auto scan, qui va s'exécuter en arrière-plan, et cherche dans le répertoire du bureau, documents, téléchargements..., afin de trouver de nouveaux fichiers pour les 'indexer' et les ajouter à la base de données.

### 3.1. Diagrammes UML :

Diagramme d'activité : Processus de recherche



### Diagramme de cas d'utilisation : Les fonctionnalités



## Annexes

L'interface principale de l'application graphique d'indexation, dont l'utilisateur peut chercher les fichiers et voir les résultats selon les

**File Indexer**  
 v1.0.0 | Full-text desktop search

Ctrl+Enter Search | Ctrl+E Export | Light

Search Index Browse History Settings

Search:  Search Clear

Ext:  Folder:  Sort: relevance Apply

| Filename                                       | Extension | Size    | Modified   | Path                                                                                                         | Score |
|------------------------------------------------|-----------|---------|------------|--------------------------------------------------------------------------------------------------------------|-------|
| 1779270571790_README.md                        | .md       | 8.6 KB  | 2026-05-20 | C:/Users/HP/Desktop/projet_assistant_chat_de_IBRAHIM/Big projects/project-manager-electronjs/backend/uploads | 6.64  |
| 1779204415056_1778690651_main.js               | .js       | 1.5 KB  | 2026-05-19 | C:/Users/HP/Desktop/projet_assistant_chat_de_IBRAHIM/Big projects/project-manager-electronjs/backend/uploads | 5.81  |
| 1778690651_main.js                             | .js       | 1.5 KB  | 2026-05-13 | C:/Users/HP/Desktop/projet_assistant_chat_de_IBRAHIM/Big projects/project-manager-electronjs/backend/uploads | 5.81  |
| 1778690917_package.json                        | .json     | 669.0 B | 2026-05-13 | C:/Users/HP/Desktop/projet_assistant_chat_de_IBRAHIM/Big projects/project-manager-electronjs/backend/uploads | 2.00  |
| 1779197324322_formulaire_2025_04_22_151831.xls | .xlsx     | 37.7 KB | 2026-05-19 | C:/Users/HP/Desktop/projet_assistant_chat_de_IBRAHIM/Big projects/project-manager-electronjs/backend/uploads | 2.00  |
| 1778691226_package.json                        | .json     | 669.0 B | 2026-05-13 | C:/Users/HP/Desktop/projet_assistant_chat_de_IBRAHIM/Big projects/project-manager-electronjs/backend/uploads | 2.00  |
| run_flask_server.py                            | .py       | 145.0 B | 2026-05-19 | C:/Users/HP/Desktop/projet_assistant_chat_de_IBRAHIM/Big projects/project-manager-electronjs/backend         | 1.00  |

< Prev Page 1/1 Next > (9 total) Hide Preview Export CSV Save Search

Preview Show in Folder

Found 9 results for 'main'

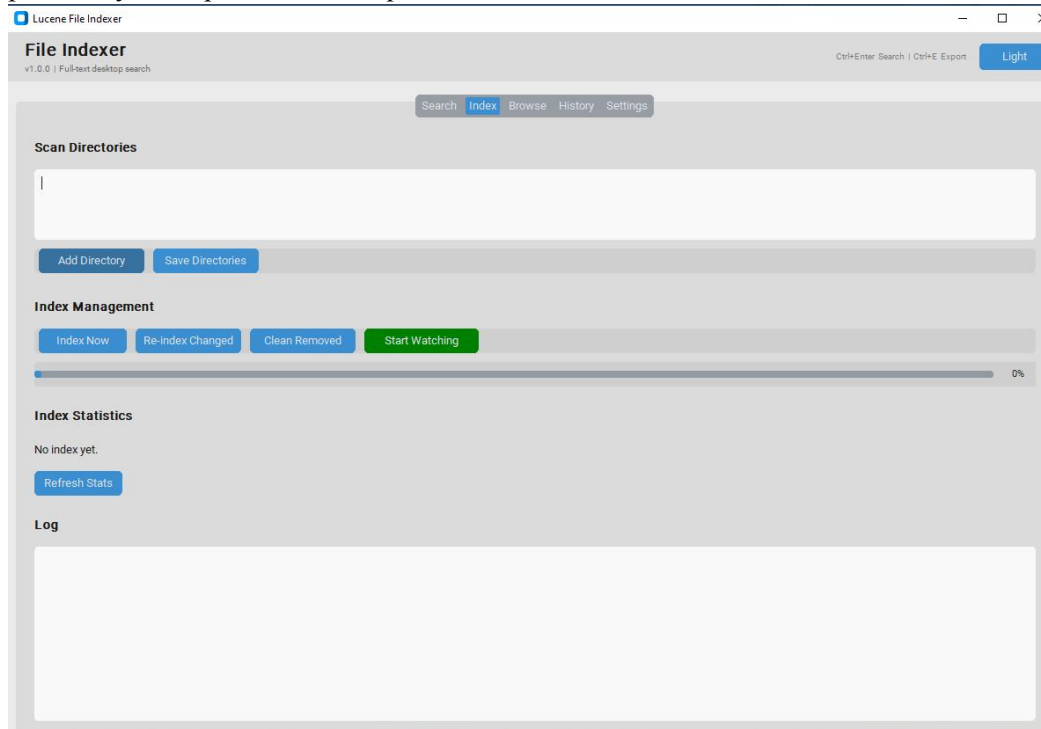
filtres.

Figure 4 : interface principale

Lors de l'affichage des résultats, type de fichier est coloré selon le réglage par défaut, l'utilisateur peut les changer après.

L'utilisateur peut aussi ajouter / indexer de nouveaux répertoires pour les inclure lors de la recherche et

peut analyser l'opération en temps réel.



Aussi que l'utilisateur peut configurer la paramètres de colorisation de chaque type de fichier, selon l'extension.

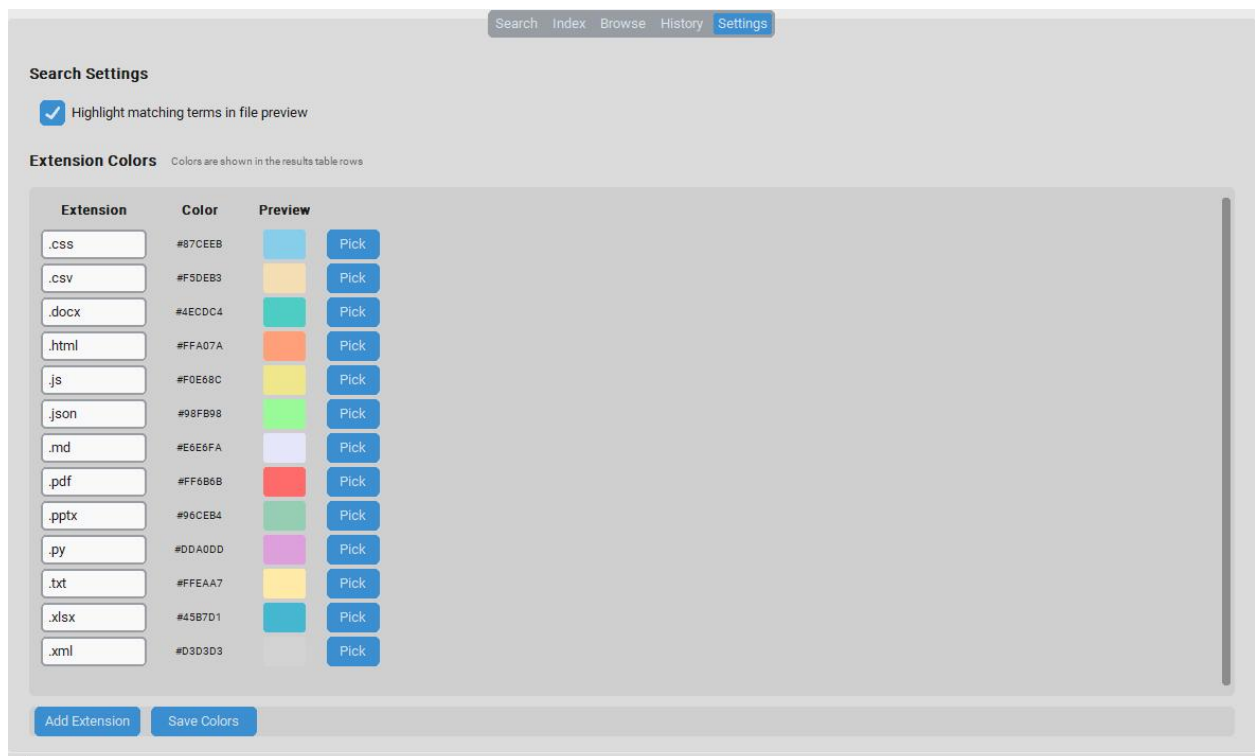


Figure 5 : Changer les couleurs par défaut

## 1. Contexte et technologies



Il existe plusieurs choix quand il s'agit de créer une application bureau, dont on a les plus populaires : JavaFX (java), ElectronJs(javascript), PyQt5(python).

On a choisi **ElectronJs** comme moteur principale de l'application desktop, car grâce à sa flexibilité dans les interfaces utilisateur, elle accepte presque tous les outils de construction du web pour développer des applications bureau, tant que : HTML, CSS, VUE, React...

Pour stocker les données, on est besoin d'un serveur et une base de données, pour stocker les différents types d'information, et assurer la gestion des API.

Le meilleur choix pour un serveur est **Flask**, car il est léger, et supports le système des rendez-vous pour les tâches.

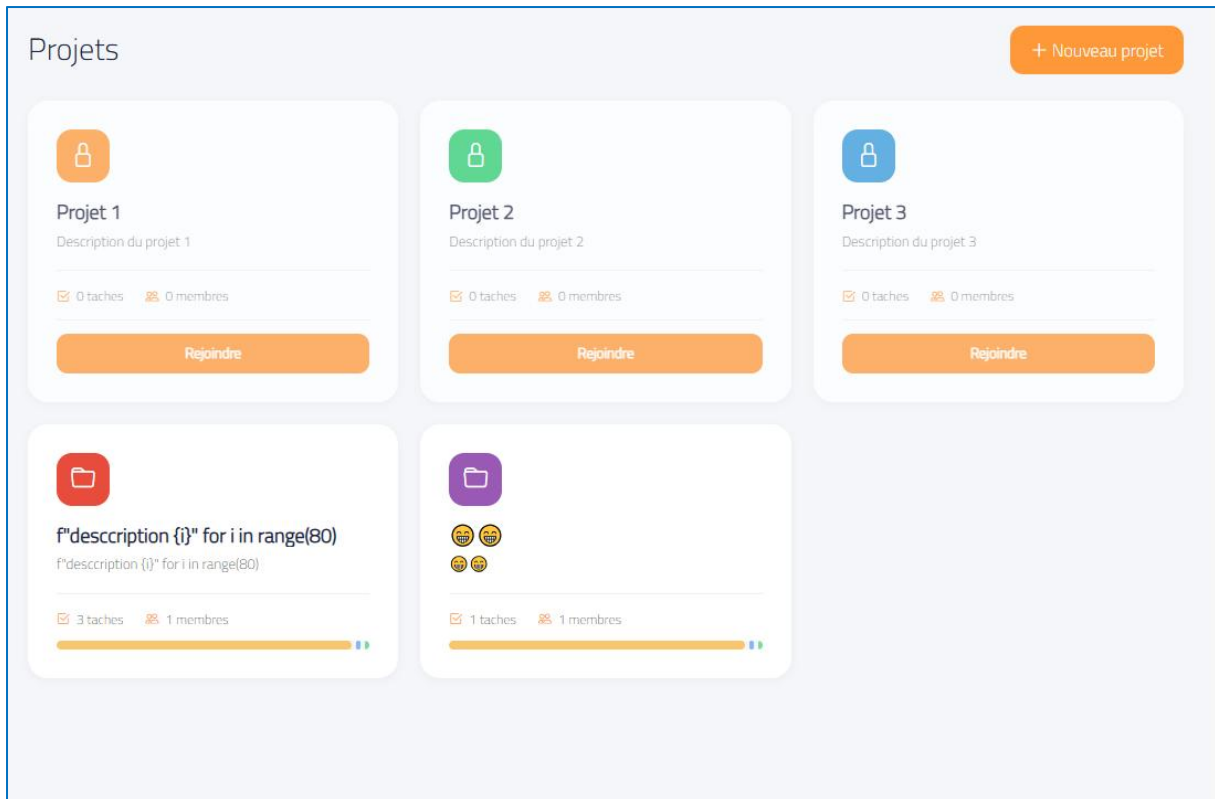
Un autre avantage de choisir Flask, c'est qu'il est plus léger que Django, mais le plus important c'est qu'il supporte l'ORM.

Il donne la possibilité d'exécuter des requêtes SQL en utilisant du concept d'Orientée  
Objet :

```
class Book(db.Model):
 id = db.Column(db.Integer, primary_key=True)
 title = db.Column(db.String(100))
 author = db.Column(db.String(50))
```

35





Ça c'est la vision de la liste des projets, les projets créés par l'utilisateur actuelle affichent des statistiques, pour les autres projets il affiche le bouton pour envoyer la demande de rejoindre le projet et avoir accès aux ses ressources.

Figure 3 : liste des projets

### 2.3. Tableau des tâches Kanban :

- Tableau à trois colonnes : À faire / En cours / Terminé
- Cartes de tâches visuelles affichant des images, des tags, le titre, la description, la date limite, les commentaires, les vues et les avatars des auteurs.

*Voici exemple de tâche :*

- Recherche en temps réel dans les titres, descriptions et tags
- Panneau de filtres multicritères : par tag, utilisateur ou projet
- Tri des tâches par date de création (du plus récent au plus ancien, ou inversement)
- Réinitialisation de tous les filtres en un seul clic



clic.

Créer des tâches dans un projet, ou bien d'importer des fichiers vers ce projet :

Retour

dateofjoin

Propriétaire

+ Nouvelle tâche

dateofjoin

4 membres

Membres (4)

Ajouter

user\_8 20 mai 2026 Propriétaire

user\_4 20 mai 2026 x

user\_10 24 mai 2026 x

user\_11 24 mai 2026 x

Tâches

Fichiers

0

Fichiers

0 B

Taille totale

Cliquez pour uploader des fichiers  
Vous pouvez sélectionner plusieurs fichiers

## 2.4. Création et gestion des tâches :

- Création de tâches avec types Simple ou Avancé

Nouvelle tâche

×

Titre

Tâche 1

Type de tâche

Avancee

Description / Notes

B

I

U

Voici un éditeur de texte dont vous pouvez écrire votre contenu de tâche selon votre besoin

- Éditeur de texte enrichi pour les descriptions (gras, italique, souligné).
- Définition des dates limites, des niveaux de priorité (Faible / Moyenne / Élevée) et du statut.

Date limite

19/05/2026

📅

Priorite

Moyenne

▼

Statut

A faire

▼

Rappel \*

22/05/2026 15:45

📅

! NB : Le rappel va passer l'exécution vers l'arrière-plan, si l'application est fermée, pour assurer tracker les rappels des tâches.

- Ajout de tags (séparés par des virgules) et d'URLs (une par ligne)
- Téléversement direct des images vers le serveur (stockées sur disque et pas sous forme de base64 volumineux dans la base de données).

Tags (separés par des virgules)

Travail, rendez-vous, others

URLs (une par ligne)

<https://medium.com/>  
<https://medium.com/system-weakness/30-one-liner-bash-scripts-that-automate-80-of-your-bug-bounty-recon-f0dcd1f5205f>

- Ajout de pièces jointes (PDF, images, archives, etc.)
- Champ de rappel obligatoire avec sélecteur de date et d'heure
- Attribution des tâches à un projet ou espace de travail spécifique.

- 40

- 41

### 3. Architecture du Système

## 4.1 Vue d'Ensemble

L'application suit une architecture en trois couches : présentation, logique métier et données.

La communication entre le client Electron et le serveur Flask s'effectue via HTTP sur le port localhost :5000 en utilisant l'API REST.

### 3.2. Composantes Principales

## Frontend (ElectronJs + VueJs)

- Shell de bureau : Electron v42
- Framework UI : Vue.js 2 (via CDN)
- Styling : Bootstrap 5 + CSS personnalisé
- Icônes : Bootstrap Icons

## Backend (Flask)

- Framework : Flask (Python)
- ORM : SQLAlchemy
- Base de données : SQLite
- CORS : Flask-cors pour la communication cross-origine

## Stockage & Communication

- Session : Stockée en JSON sur le disque (user-session. Son)
- Fichiers utilisateur : Dossier 'uploads' et 'images'
- Communication: ketch API via IPC bridge Electron

### 3.3. Spécifications Technique

### 3.3.1 Stack Technologique Détaillé

| Couche          | Technologie            | Version/Notes    |
|-----------------|------------------------|------------------|
| Desktop         | Electron               | V42.0.1          |
| Framework UI    | Vue.js                 | V2 (CDN)         |
| CSS             | Bootstrap + Custom CSS | V5               |
| Backend         | Flask                  | Python 3.8+      |
| ORM             | SQLAlchemy             | Lattes           |
| Base de Données | SQLite                 | Fichier test. DB |
| Diagrammes      | Chartjs & Frappe Gantt | Dernière         |



- 44

## 6. Défis Techniques

| Défi                         | Description                                                           | Solution                                                        |
|------------------------------|-----------------------------------------------------------------------|-----------------------------------------------------------------|
| Communication Client-Serveur | Assurer la communication stable entre Electron et Flask sur localhost | Utilisation de fetch API avec IPC bridge et gestion des erreurs |
| Gestion Session              | Persister la session entre redémarrages d'application                 | Stockage JSON local avec chiffrement optionnel                  |
| Téléchargement Fichiers      | Gérer l'upload et l'affichage d'images et fichiers                    | Encodage base64 pour images, gestion des dossiers uploads       |
| Intégrité Données            | Assurer la cohérence des données entre client et serveur              | Validation côté serveur et gestion des transactions             |

## 7. Récapitulatif

## 7.1. Réalisations

Ce projet a permis de développer une application complète et fonctionnelle de gestion de projets combinant les meilleures pratiques en ingénierie logicielle. Les objectifs initialement fixés ont été largement atteints :

- Création d'une application de bureau moderne utilisant Electron.js
- Implémentation d'un serveur backend robuste avec Flask et SQLAlchemy
- Développement d'une interface utilisateur intuitive avec Vue.js et Bootstrap
- Mise en place d'un système complet de gestion des utilisateurs, projets et tâches
- Gestion efficace des défis techniques rencontrés lors du développement

## 7.2. Enseignements et Apprentissages

Ce projet a fourni des apprentissages précieux :

- Développement d'une application complète
- Compréhension approfondie de l'architecture serveur-client
- Expérience dans l'intégration de multiples technologies et Frameworks.
- Résolution de problèmes complexes dans un contexte technique réaliste

### 7.3. Perspectives et Améliorations Futures

Plusieurs améliorations et extensions pourraient être envisagées :

1. Déploiement multi-plateforme : Support pour MacOS et Linux aussi
2. Application mobile : Développement d'une version iOS/Android, qui supporte les notifications et la synchronisation cloud.
3. Notifications en temps réel : Web Sockets pour les mises à jour instantanées
4. Exportation de rapports : Génération de rapports PDF/Excel.



## Conclusion générale

La réalisation de ces projets m'a permis de comprendre de nouvelles technologies, savoir de nouvelles méthodes et algorithmes pour traiter des données de manière simple, aussi savoir manipuler les données en niveau bas, traitement intelligent, etc. Ce qui aussi importe le développement des applications bureau, développement web, implémentation des modèles d'intelligence artificielle pour résoudre des problèmes ou pour simplifier et effectuer des tâches précises.

L'implémentation du chatbot assistant basé sur RAG m'a permis de comprendre les mécanismes avancés du traitement du langage naturel et l'importance de la récupération d'information augmentée. J'ai appris à configurer des modèles pré-entraînés comme Mistral 7B, à structurer des données en vecteurs, et à construire des pipelines efficaces reliant un modèle d'IA à une base de connaissances. Cet apprentissage m'a montré comment les entreprises peuvent tirer profit de l'IA sans ressources informatiques massives.

Une autre chose, c'est que la mise en place du moteur d'indexation avec Lucene et Apache Tika m'a initié aux principes fondamentaux de la recherche et de l'indexation d'informations. Cette expérience souligne l'importance de l'optimisation des performances dans les systèmes traitant de grandes masses de données.

Finalement le développement de l'application de gestion de projets m'a donné une vision claire sur le cycle de vie d'une application, de l'architecture système à l'intégration de multiples technologies (Electron, Vue.js, Flask, SQLAlchemy,). J'ai compris les défis réels de la conception interface-utilisateur et la nécessité d'une architecture bien structurée.

# Bibliographie

- [1] [Google Colab](#)
- [2] [Hugging face](#)
- [3] [Data Maroc](#) :
- [4] [GPU](#)
- [5] [RAG tutorial](#)
- [6] [About LLMs](#)
- [7] [About RAG](#)
- [8] [qu'est-ce un système décisionnel](#)
- [9] [Lucida Chart pour créer les diagrammes](#)
- [10] [Lien github projet assistant](#)
- [11] [Lien github projet indexeur de fichiers](#)
- [12] [Lien github Project manager](#)